Practical 3: MPI File Transfer Report

Le Quy Ton
23BI14425

December 15, 2025

1 Task Overview

The objective of this practical work is to upgrade a previously implemented TCP file transfer system into a system utilizing the Message Passing Interface (MPI) standard. The implementation is done in Python using the `mpi4py` library.

2 Choice of MPI Implementation

I chose `mpi4py` (Python bindings for MPI) for the following reasons:

- **Ease of Use:** Python allows for rapid development and simpler handling of file I/O and string manipulations compared to C/C++.
- **Direct Mapping to MPI Standard:** `mpi4py` provides a close mapping to standard MPI primitives (e.g., `MPI_Send`, `MPI_Recv`, `MPI_Probe`), ensuring the design principles remain true to MPI concepts.
- **Ecosystem Integration:** It integrates seamlessly with the broader Python scientific ecosystem if further data processing is needed later.

3 System Organization

The system is organized in a centralized "star" topology, adopting a Client-Server model tailored for MPI ranks.

- **Rank 0 (The Receiver/Server):** This process acts as the central repository. It determines the number of potential senders and enters a listening state, ready to receive files from any other rank.
- **Rank 1 to N (The Senders/Clients):** These processes identify their assigned file from command-line arguments, initiate connection to Rank 0, and push their data.

Figure 1 illustrates this organization where multiple sender ranks transmit data to the single receiver rank.

4 Service Design and Protocol

The file transfer service is designed around a strictly ordered, metadata-first protocol to ensure robust transmission. To handle potentially large files, data is transmitted in fixed-size chunks (4096 bytes).

The communication sequence between a Sender (Rank i) and the Receiver (Rank 0) is as follows:

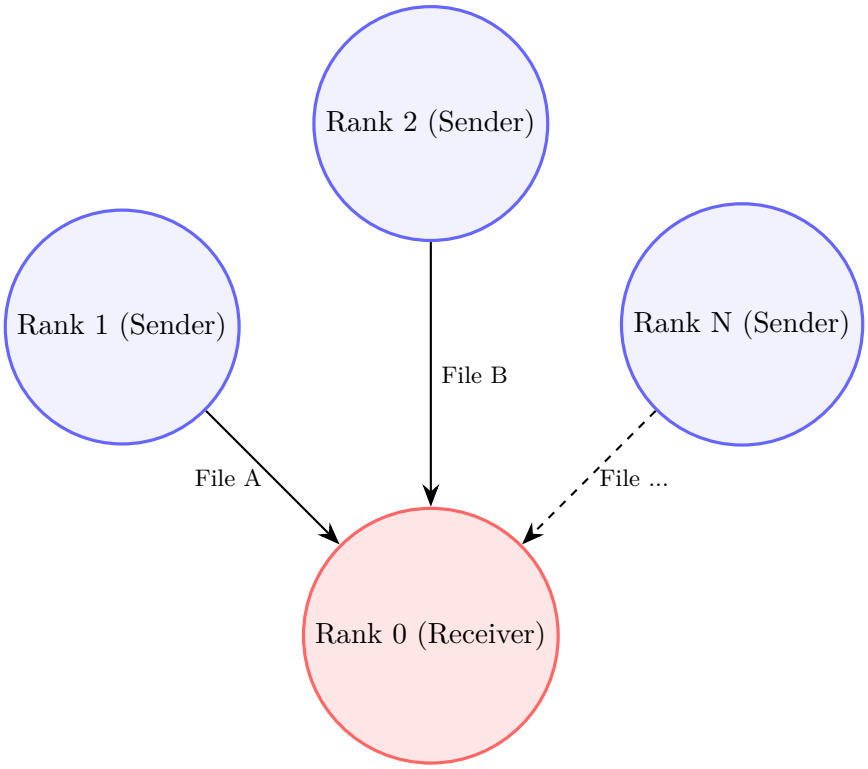

Figure 1: System Organization: Multiple Senders pushing files to Rank 0.

1. **Metadata Header:** The sender first transmits essential file metadata: filename length, filename string, and total file size.
2. **Data Payload:** The file content is read in chunks and sent sequentially using a specific `TAG_DATA`.
3. **Completion Signal:** Once all data is sent, a final zero-byte message with `TAG_DONE` is sent to signal the end of the transfer.

Figure 2 depicts the timeline of messages exchanged during a single file transfer session.

5 Implementation Details

The implementation uses standard blocking MPI calls. A key aspect of the receiver implementation is the use of `comm.Probe()`. Since data is arriving in unknown chunk sizes (up to 4096 bytes) and the stream is terminated by a specific tag, the receiver probes incoming messages to determine their size and tag before allocating a buffer and committing to a receive operation.

Below are the critical code snippets illustrating the core logic.

5.1 Sender Implementation Snippet

The sender reads the file in chunks and uses the uppercase `comm.Send` for efficient transmission of raw byte buffers.

```
# (Inside sender function)
# send data in chunks
with open(filepath, 'rb') as f:
    sent = 0
    while sent < filesize:
        chunk = f.read(CHUNK)
```

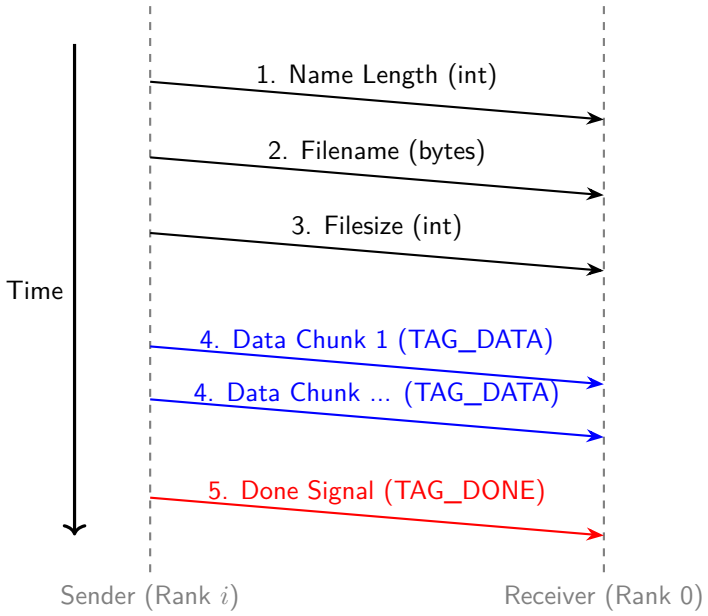

Figure 2: Sequence Diagram of the MPI File Transfer Protocol.

```
        if not chunk:
            break
        # send raw bytes
        comm.Send([chunk, MPI.BYTE], dest=dest, tag=TAG_DATA)
        sent += len(chunk)
# notify done
comm.send(True, dest=dest, tag=TAG_DONE)
print(f"[rank {rank}][SENDER] Sent '{fname}' ({filesize} bytes) to rank {dest}."
    , flush=True)
```

Listing 1: Core sender loop transmitting file data.

5.2 Receiver Implementation Snippet

The receiver loops until the total expected bytes are received. It uses `Probe` to dynamically handle incoming data chunks or the completion signal.

```
# (Inside receiver function loop)
received = 0
with open(out_name, 'wb') as out:
    while received < filesize:
        # Probe for next incoming message
        status = MPI.Status()
        comm.Probe(source=expected_from, tag=MPI.ANY_TAG, status=status)
        t = status.Get_tag()
        if t == TAG_DATA:
            count = status.Get_count(MPI.BYTE)
            buf = bytearray(count)
            comm.Recv([buf, MPI.BYTE], source=expected_from, tag=TAG_DATA,
    status=status)
            out.write(buf)
            received += count
        elif t == TAG_DONE:
            # consume the done message and break
            _ = comm.recv(source=expected_from, tag=TAG_DONE)
            break
        else:
```

```
            # unexpected tag; try to consume and continue
            comm.recv(source=expected_from, tag=t)
```

Listing 2: Receiver loop using Probe to handle incoming chunks.

6 How to Run

The program requires an MPI runtime (e.g., OpenMPI or MPICH) and the `mpi4py` library. Rank 0 is automatically designated as the receiver.

To transfer two files using 3 processes (1 receiver, 2 senders):

```
mpirun -np 3 python3 mpi_transfer.py send file1.txt file2.txt
```

Rank 1 will send `file1.txt`, and Rank 2 will send `file2.txt`. The receiver will save them with prefixes indicating their source rank.